



Документация

Хакатон «Цифровой суверенитет 4.0»

Кейс «Звездный час администратора безопасности UserGate NGFW»

Команда «Краники»

Ссылка на GITHUB <https://github.com/SALAT1Ks/UserGateHaha>

Состав команды:

*Андреев Дмитрий Олегович
Богословский Мирослав Анатольевич
Попов Егор Федорович*

Оглавление

Термины	3
Топология сети и карта сети	4
NAT/DNAT описание	5
Первое правило	5
Второе правило	5
Межсетевой экран.....	6
Первое правило	6
Второе правило	6
Профиль COB.....	7
Сигнатуры COB	8

Термины

COB – система обнаружение вторжения, модуль NGFW

NGFW (Next-Generation Firewall – Межсетевой экран нового поколения) – это DPI (Deep Packet Inspection – Глубокая проверка пакетов)

Аномальный трафик – это любые действия, которые не похожи на обычное поведение пользователя. Это могут быть попытки взлома веб-сервера: SQL-инъекции, попытки подбора паролей или эксплуатация уязвимостей в коде сайта.

Топология – физическое или логическое расположение узлов компьютерной сети относительно друг друга и способы их соединения.

Топология сети – это схема расположения устройств (компьютеров, принтеров, роутеров) и способ их соединения друг с другом.

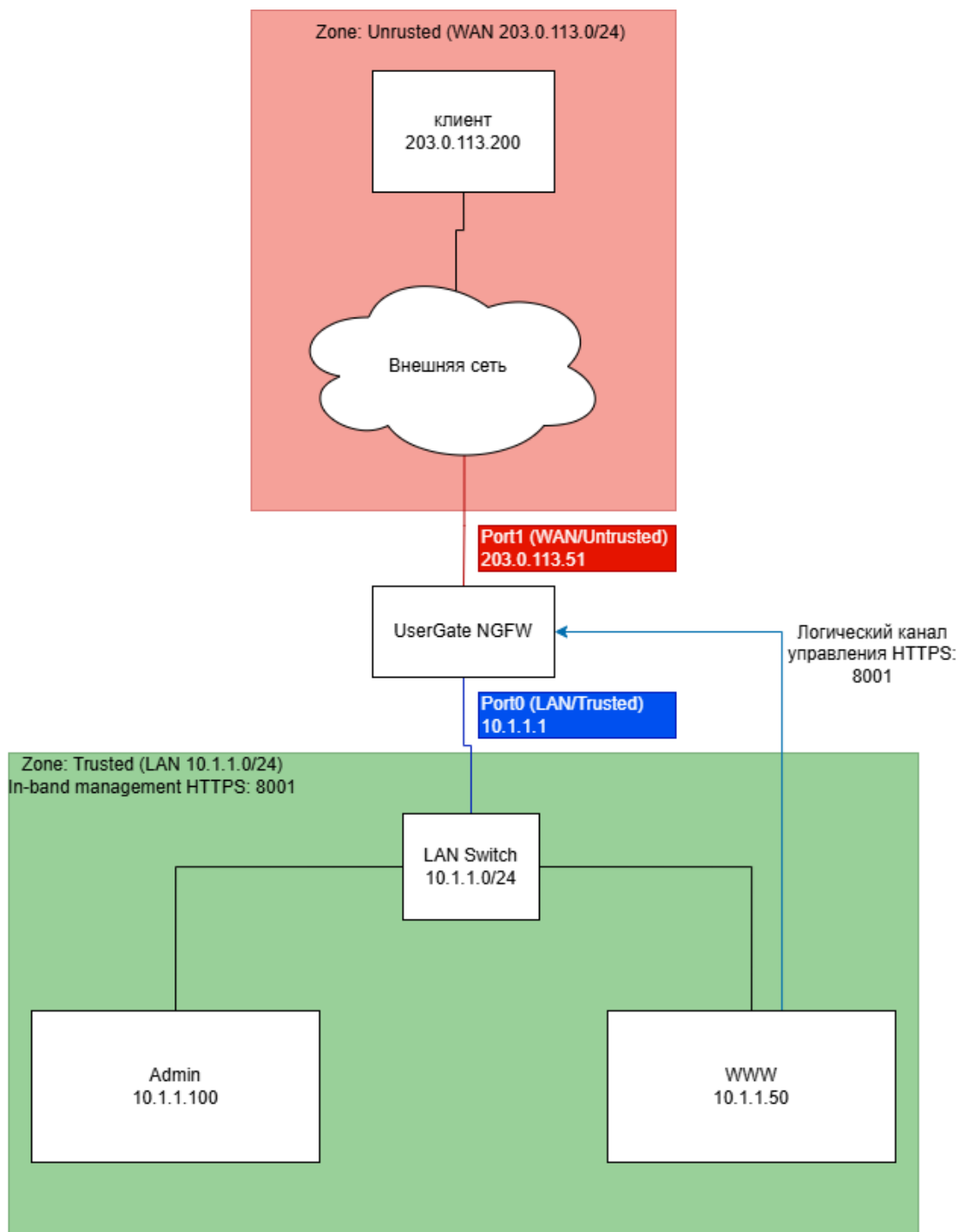
Топология сети – это принцип или архитектура того, как устройства соединены между собой (например, «звезда», «кольцо» или «ячеистая» структура). Это математическая и логическая модель связей.

Карта сети – это конкретная схема или визуализация вашей инфраструктуры. На карте отображаются реальные узлы (серверы, роутеры), их марки, IP-адреса и то, как они физически или логически взаимодействуют друг с другом на данный момент.

NAT – Преобразование сетевых адресов. Обычно используется для разрешения доступа из локальной сети в интернет

DNAT – Перенаправляет трафик на указанный IP-адрес. Обычно используется для публикации внутренних ресурсов в интернет

Топология сети и карта сети



NAT/DNAT описание

Зона источника – трафик, летящий из внешнего интернета/от атакующего.

Адрес источника – чтобы ловить вообще всех сканеров со всех левых IP.

Зона назначения – зависит от того, куда привязан твой внешний IP

Адрес назначения – указывается конкретно внешний IP-адрес самого межсетевого экрана.

Сервис – порты, по которым будет работать правило.

Первое правило

Название – Dnat_bridged_kraniki

Тип – DNAT

Зона источника – Untrusted

Адрес источника – Любой

Адрес назначения – WWW (203.0.113.51)

Сервис – HTTP; HTTPS

Адрес назначения DNAT – 10.1.1.50

Входящий поток данных, поступающий на внешний адрес периметра 203.0.113.51 по порту 80, 443/tcp, транслируется на локальный адрес веб-сервера 10.1.1.50 (порт 80, 443/tcp).

Второе правило

Название – NAT from Trusted to Untrusted

Тип – NAT

Зона источника – Management; Trusted

Адрес источника – Любой
Зона назначения – Untrusted
Адрес назначения – Любой
Сервис – Любой

NAT из доверенной в ненадежную Правило NAT, которое позволяет перенаправлять трафик из доверенной зоны в ненадежную. Это пример правила, которое при необходимости можно изменить или удалить

Межсетевой экран

Первое правило

Название – firewall_kraniki
Действие – разрешить
Профиль COB – COB_kraniki
Зона источника – Untrusted
Адрес источника – любой
Зона назначения – Management
Адрес назначения – WWW-INT (10.1.1.50); www-int-list (10.1.1.0/24)
Сервис – HTTP; HTTPS; enable_ephemeral (tcp (1-65535), udp (1-65535))

Правило брандмауэра, которое разрешает веб-трафик (HTTP/HTTPS) из ненадежной зоны (Untrusted) в зону управления (Management) к серверу WWW-INT с обязательной проверкой профилем COB.

Второе правило

Название – Allow trusted to untrusted

Действие – разрешить

Профиль COB – COB_kraniki

Зона источника – Management; Trusted

Адрес источника – любой

Зона назначения – Untrusted

Правило брандмауэра, которое разрешает трафик из доверенной зоны в ненадежную.

Профиль COB

Название – COB_kraniki

Фильтр – Любые, самописные сигнатуры COB

Проверка входящих пакетов на вредоносную активность.

Сигнатуры COB

1. Block_Nmap_Web

```
UASL(  
    .name = "Block_Nmap_Web";  
    .dst_port = 80;  
    .pattern = "Nmap";  
    .no_case;  
)
```

Выявляет сканер Nmap по заголовкам сетевых запросов, пресекая первичный сбор информации о веб-сервере.

Использует паттерн Nmap (без учета регистра).

Пример паттерна в пакете User-Agent: Mozilla/5.0 (compatible; Nmap Scripting Engine; <https://nmap.org/book/nse.html>)

2. Block_Nikto_Scanner

```
UASL(  
    .name = "Block_Nikto_Scanner";  
    .service = http;  
    .pattern = "Nikto";  
    .no_case;  
)
```

Реагирует на запросы утилиты Nikto, блокируя автоматизированный поиск устаревших скриптов и ошибок конфигурации.

Использует паттерн Nikto (без учета регистра)

Пример паттерна в пакете User-Agent: Nikto/2.1.6 (EPE/0.2.6)

3. Block_Nmap_NSE_Scanner

```
UASL(  
    .name = "Block Nmap NSE Scanner";  
    .dst_port = 80;  
    .pattern = "Nmap Scripting Engine";  
    .no_case;  
)
```

Фиксирует использование скриптового движка Nmap, отсекая запуск сложных автоматических проверок на уязвимости.

Использует паттерн Nmap Scripting Engine (без учета регистра)

Пример паттерна в пакете User-Agent: Mozilla/5.0 (compatible; Nmap Scripting Engine; <https://nmap.org/book/nse.html>)

4. Block_SQLi_Union

```
UASL(  
  .name = "Block_SQLi_Union";  
  .service = http;  
  .pattern = "union select";  
  .no_case;  
)
```

Обнаруживает попытки проведения инъекций через оператор объединения для защиты баз данных от несанкционированных выгрузок.

Использует паттерн union select (без учета регистра)

Пример паттерна в пакете

```
GET /index.php?id=1%20union%20select%201,2,user() HTTP/1.1
```

5. Block_SQLi_Auth_Bypass

```
UASL(  
  .name = "Block_SQLi_Auth_Bypass";  
  .service = http;  
  .pattern = "' or 1=1";  
  .no_case;  
)
```

Распознает логические конструкции, применяемые злоумышленниками для авторизации в системе без знания реального пароля.

Использует паттерн ' or 1=1 (без учета регистра)

Пример паттерна в пакете

```
username=admin'%20or%201%3D1&password=any
```

6. Block_OWASP_LFI_Etc_Passwd

```
UASL(  
  .name = "Block_OWASP_LFI_Etc_Passwd";  
  .service = http;  
  .pattern = "/etc/passwd";  
  .no_case;  
)
```

Пресекает атаки локального включения файлов, не давая атакующему прочитать системный файл с пользователями.

Использует паттерн `/etc/passwd` (без учета регистра)

Пример паттерна в пакете GET `/vuln.php?page=../../../../etc/passwd HTTP/1.1`

7. Block_OWASP_RCE_Linux_Whoami

```
UASL(  
  .name = "Block_OWASP_RCE_Linux_Whoami";  
  .service = http;  
  .pattern = "; whoami";  
  .no_case;  
)
```

Отслеживает внедрение системных команд. Срабатывает при попытке хакера узнать свои текущие привилегии в операционной системе.

Использует паттерн `; whoami` (без учета регистра)

Пример паттерна в пакете GET `/ping.php?ip=127.0.0.1%3B%20whoami HTTP/1.1`

8. Block_XSS_Script_Tag

```
UASL(  
  .id = 1000009;  
  .name = "Block_XSS_Script_Tag";  
  .dst_port = 80;  
  .pattern = "<script>";  
  .no_case;  
)
```

Моментально сбрасывает соединение при обнаружении открывающего тега скрипта, защищая ресурс от внедрения произвольного кода.

Использует паттерн `<script>` (без учета регистра)

Пример паттерна в пакете GET `/search.php?q=%3Cscript%3E HTTP/1.1`

9. Block_XSS_Event_Handler

```
UASL(  
  .id = 1000010;
```

```

        .name = "Block_XSS_Event_Handler";
        .service = http;
        .pattern = "onerror=";
    )

```

Вычисляет скрытые атаки межсайтового скриптинга, когда вредоносная нагрузка прячется прямо в обработчиках событий страницы.

Использует паттерн onerror= (с учетом регистра)

Пример паттерна в пакете

GET /profile?name=<img%20src=x%20onerror=alert(1)> HTTP/1.1

10. Block_OWASP_XXE_Entity_Declaration

```

UASL(
    .name = "Block_OWASP_XXE_Entity_Declaration";
    .service = http;
    .pattern = "<!ENTITY";
    .no_case;
)

```

Контролирует парсинг документов. Блокировка внешних сущностей спасает от чтения локальных файлов и сканирования сети.

Использует паттерн <!ENTITY (без учета регистра)

Пример паттерна в пакете

<?xml version="1.0"?><!DOCTYPE test [<!ENTITY xxe SYSTEM "file:///etc/passwd">]><root>&xxe;</root>

11. Block_Nmap_URL

```

UASL(
    .name = "Block_Nmap_URL";
    .service = http;
    .host = "nmap.org";
)

```

Фильтрует трафик на наличие ссылок на официальный ресурс сканера, которые часто оставляют автоматические инструменты при проверках.

Использует паттерн nmap.org (в поле хоста/ссылок)

Пример паттерна в пакете Referer: <https://nmap.org/book/nse.html>

12. Block_Testing_Purposes

```
UASL(  
    .name = "Block_Testing_Purposes";  
    .service = http;  
    .pattern = "TESTING_PURPOSES_ONLY";  
    .no_case;  
)
```

Отлавливает специальные отладочные маркеры тестового трафика, что полезно для фильтрации проверок на соревнованиях.

Использует паттерн TESTING_PURPOSES_ONLY (без учета регистра)

Пример паттерна в пакете X-Check-Header: TESTING_PURPOSES_ONLY

13. Block_DVWA_XSS

```
UASL(  
    .name = "Block_DVWA_XSS";  
    .service = http;  
    .pattern =  
    "/dvwa/vulnerabilities/xss_r/?name=<script>alert('XSS')</script>";  
    .no_case;  
)
```

Точечная сигнатура под уязвимое приложение, прерывающая конкретный вектор атаки через параметры адресной строки.

Использует паттерн

/dvwa/vulnerabilities/xss_r/?name=<script>alert('XSS')</script>
> (без учета регистра)

Пример паттерна в пакете GET

/dvwa/vulnerabilities/xss_r/?name=%3Cscript%3Ealert('XSS')%3C/
script%3E HTTP/1.1

14. Block_Scan_Robots_Txt

```
UASL(  
    .name = "Block_Scan_Robots_Txt";  
    .service = http;  
    .pattern = "GET /robots.txt";  
    .no_case;  
)
```

Регистрирует массовые обращения к файлу политик сайта, помогая вычислить ботов в поисках скрытых директорий.

Использует паттерн `GET /robots.txt` (без учета регистра)

Пример паттерна в пакете `GET /robots.txt HTTP/1.1`

15. Block_Scan_HNAP1

```
UASL(  
    .name = "Block_Scan_HNAP1";  
    .service = http;  
    .pattern = "GET /HNAP1";  
    .no_case;  
)
```

Останавливает сканирование домашнего сетевого оборудования по известному уязвимому протоколу управления роутерами.

Использует паттерн `GET /HNAP1` (без учета регистра)

Пример паттерна в пакете `GET /HNAP1/ HTTP/1.1`

16. Block_Scan_Evox_About

```
UASL(  
    .name = "Block_Scan_Evox_About";  
    .service = http;  
    .pattern = "GET /evox/about";  
    .no_case;  
)
```

Защищает интерфейсы систем интернет-телефонии, блокируя сбор данных о текущих версиях оборудования.

Использует паттерн `GET /evox/about` (без учета регистра)

Пример паттерна в пакете `GET /evox/about HTTP/1.1`

17. Block_RCE_Uname_A

```
UASL(  
    .name = "Block_RCE_Uname_A";  
    .service = http;  
    .pattern = "uname%20-a";  
    .no_case;  
)
```

Запрещает передачу команды для вывода версии ядра системы, что обычно предшествует загрузке целевого эксплоита.

Использует паттерн `uname%20-a` (без учета регистра)

Пример паттерна в пакете `GET /shell.php?run=uname%20-a HTTP/1.1`

18. Block_RCE_List_Directory

```
UASL(  
    .name = "Block_RCE_List_Directory";  
    .service = http;  
    .pattern = ";ls";  
    .no_case;  
)
```

Срабатывает на команды листинга директорий, мешая злоумышленнику осмотреться на сервере после успешного взлома.

Использует паттерн `;ls` (без учета регистра)

Пример паттерна в пакете `GET /index.php?dir=%3Bls HTTP/1.1`

19. Block_TCP_Empty_Payload

```
UASL(  
    .protocol = tcp;  
    .tcp.flags = S;  
    .track = src_ip;  
    .nopayload;  
)
```

Выявляет пустые пакеты транспортного уровня, характерные для скрытого сканирования открытых сетевых портов.

Использует паттерн `.tcp.flags=S` (флаг SYN), `.nopayload` (пустой пакет).

Пример паттерна в пакете `Flags: 0x002 (SYN), а длина поля TCP Segment Len = 0.`

20. Block_XML_Parser

```
UASL(  
    .name = "Block_XML_Parser";  
    .service = http;  
    .pattern = "xml_parser.php";
```

```
.no_case;  
)
```

Жестко ограничивает доступ к уязвимому файлу обработки данных, исключая саму возможность эксплуатации ошибок разбора текста.

Использует паттерн `xml_parser.php` (без учета регистра)

Пример паттерна в пакете `GET /vendor/xml_parser.php HTTP/1.1`

21. Block_Vulnerable_Pattern

```
UASL(  
  .name = "Block_Vulnerable_Pattern";  
  .service = http;  
  .pattern = "vulnerable";  
  .no_case;  
)
```

Отсекает запросы с характерным словом-маркером уязвимости, эффективно справляясь с базовыми атакующими ботами.

Использует паттерн `vulnerable` (без учета регистра)

Пример паттерна в пакете `GET /vulnerable_service_test HTTP/1.1`

22. Block_Vulnerability_Folder

```
UASL(  
  .name = "Block_Vulnerability_Folder";  
  .service = http;  
  .pattern = "/vulnerabilities/";  
  .no_case;  
)
```

Перекрывает доступ к учебным каталогам стенда, не давая атакующему даже инициировать взаимодействие с опасными модулями.

Использует паттерн `/vulnerabilities/` (без учета регистра)

Пример паттерна в пакете `GET /dvwa/vulnerabilities/brute/ HTTP/1.1`

23. Block_XML_XInclude_Attack

```
UASL(  
  .name = "Block_XML_XInclude_Attack";  
  .service = http;  
  .pattern = "xi:include";  
  .no_case;
```

)

Предотвращает XInclude Injection обход фильтров XXE через включённый по умолчанию механизм XInclude для чтения файлов ОС.

Использует паттерн `.pattern="xi:include";`

Пример паттерна в пакете

```
<foo xmlns:xi="http://www.w3.org/2001/XInclude">  
  <xi:include parse="text" href="file:///etc/passwd"/>  
</foo>
```

Все правила работают по TCP и UDP, без явного указания, согласно документации UserGate https://docs.usergate.com/modifikatory-oblasti-poiska_890.html